Compte rendu de TP

Convertisseur de devises Android

Kotlin - ConstraintLayout - TableLayout - View Binding

Etudiante : Hanine Mjahed
Enseignant : M. Tahar Haouet
Date : 18 octobre 2026
Depot GitHub : https://github.com/haninemjahed-lgtm/CalculDevise

1. Introduction

L'objectif de ce TP est de developper en Kotlin une application Android, *Currency Converter*, qui convertit un montant d'une devise vers une autre a partir de taux de change fixes. L'application tient sur un seul ecran et met en pratique les notions des chapitres 5 (layouts) et 6 (widgets de selection) : un **ConstraintLayout** pour l'ecran, un **TableLayout** pour le clavier numerique, deux **Spinner** alimentes par un **ArrayAdapter**, le **View Binding**, la gestion des clics et un **Toast** pour les erreurs de saisie.

2. Choix d'interface

L'ecran est coupe en deux zones par une **Guideline horizontale a 45 %**. Ce choix garantit que la proportion demandee par la maquette est respectee quelle que soit la hauteur de l'appareil.

- **Zone du haut** : un ConstraintLayout imbrique (id `zoneHaut`) sur fond turquoise #2BB8E8, avec coins inferieurs arrondis et une legere elevation. Il contient le titre, les deux Spinner, les montants `tvMontant` et `tvResultat` (TextView 48sp, alignes a droite, soulignes), le bouton d'inversion et le bouton Convertir.
- **Contraintes** : chaque vue est reliee sur les deux axes. Les montants utilisent une largeur de 0dp (*match constraints*) entre le Spinner et le bord droit, et sont alignes verticalement sur leur Spinner.
- **Clavier** : un TableLayout de 4 TableRow x 3 colonnes. Chaque ligne a une hauteur de 0dp et un poids de 1, ce qui donne quatre lignes egales ; `stretchColumns="*"` donne trois colonnes egales.
- **Style** : un style `Touche` (bouton sans bordure, 32sp, #1BA6C9) factorise les 12 touches ; la touche C surcharge la couleur (#F0727A).
- **Ressources** : aucune valeur en dur dans le XML. Textes dans `strings.xml`, couleurs dans `colors.xml`, dimensions dans `dimens.xml`.
- **Bouton Convertir** : le theme etant Material, le bouton est un MaterialButton ; la forme en pilule est obtenue avec `backgroundTint` et `cornerRadius`.
- **Manifeste** : orientation bloquee en portrait avec `screenOrientation="portrait"`.

3. Logique de l'application

3.1 Remplissage des listes

Les devises (USD, EUR, TND, GBP, JPY, CAD) sont declarees dans un *string-array* nomme `devises`. Un meme ArrayAdapter, cree avec `createFromResource`, alimente les deux Spinner. Par defaut la source affiche USD et la cible EUR, grace a `setSelection`.

3.2 Clavier numerique

La fonction `ajouterCaractere(c)` est appelee par les touches 0 a 9 et le point. Elle respecte les regles de gestion :

- le montant initial vaut "0" et le premier chiffre tape le remplace ;
- un seul point decimal est accepte, une deuxieme pression est ignoree ;
- le montant est limite a 10 caracteres.

La touche C remet le montant a "0" et vide le resultat.

3.3 Conversion par l'euro

Les taux sont stockes dans une `Map<String, Double>`, exprimes pour 1 EUR. La conversion passe par l'euro, devise pivot :

```
montant en EUR = montant / taux(source)
resultat       = montant en EUR x taux(cible)
```

```
private fun convertir(montant: Double, source: String, cible: String): Double {
    val montantEnEuro = montant / taux.getValue(source)
    return montantEnEuro * taux.getValue(cible)
}
```

Le resultat est affiche avec `String.format(Locale.US, "%.2f", resultat)`. Le choix de `Locale.US` garantit un point decimal quelle que soit la langue du telephone. Si la source et la cible sont identiques, le calcul redonne naturellement le montant, sans cas particulier.

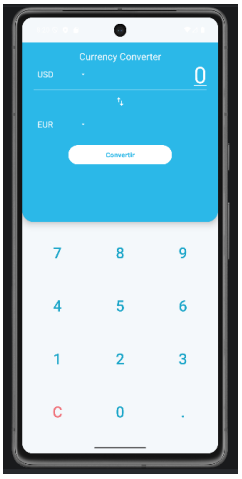
3.4 Erreurs et inversion

Si le montant vaut 0, le clic sur Convertir affiche le Toast "Veuillez saisir un montant". Le bouton d'inversion echange les positions des deux Spinner avec `setSelection`, puis relance la conversion uniquement si un resultat est deja affiche. La saisie se fait exclusivement avec le clavier de l'application : les montants sont des TextView, donc le clavier systeme n'apparait jamais.

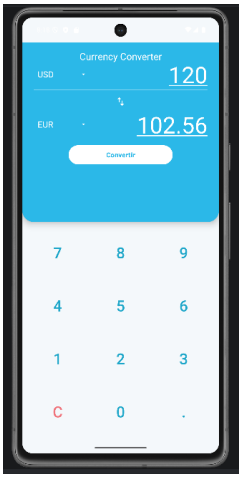
4. Tests

Les quatre cas du sujet ont ete executes sur l'emulateur Pixel 7 (API 36). Les resultats obtenus sont identiques aux resultats attendus.

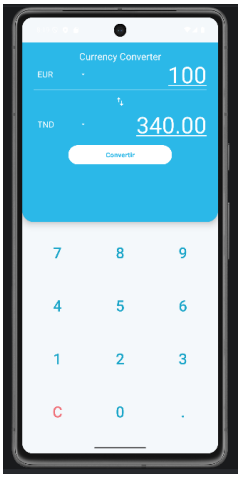
Montant	Source	Cible	Calcul	Attendu	Obtenu
120	USD	EUR	120 / 1.17	102.56	102.56
100	EUR	TND	100 x 3.40	340.00	340.00
50	GBP	USD	50 / 0.87 x 1.17	67.24	67.24
10	EUR	EUR	identique	10.00	10.00



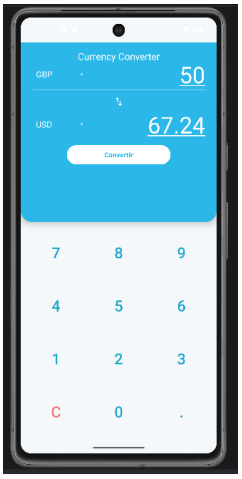
Demarrage (USD / EUR)



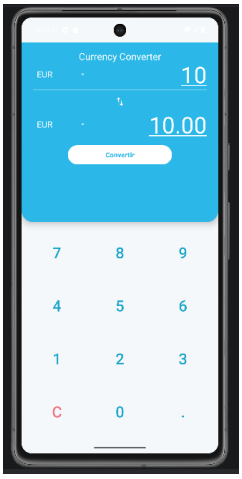
120 USD vers EUR = 102.56



100 EUR vers TND = 340.00



50 GBP vers USD = 67.24



10 EUR vers EUR = 10.00

Regles de gestion verifiees : deuxieme point ignore, saisie limitee a 10 caracteres, Toast si le montant vaut 0, touche C (montant a 0 et resultat vide), inversion des devises avec recalcul du resultat.

5. Difficultes rencontrees

- **Erreur View Binding (52 erreurs de compilation).** L'import `ActivityMainBinding` et la ligne `package` utilisaient un package different du *namespace* du projet. Apres correction du package, il ne restait que l'erreur `Unresolved reference 'R'`, resolue par l'import de la classe R du projet.
- **Ancienne application dans l'emulateur.** L'emulateur affichait un ancien projet (calcul de l'IMC) et l'etat sauvegarde faussait les devises par defaut. Une desinstallation puis un nouveau lancement ont regle le probleme.
- **Emulateur deja lance.** Le message "Pixel 7 is already running" a ete resolu par un *Cold Boot* depuis le Device Manager.
- **Design.** Le bouton Convertir ignorait `android:background` a cause du theme Material ; il a fallu utiliser `backgroundTint` et `cornerRadius`.
- **Gestion du depot Git.** Le terminal etait ouvert hors du dossier du projet, ce qui provoquait l'erreur *not a git repository* ; il a fallu se placer dans le bon dossier avant `git init`.

6. Conclusion

Ce TP a permis de combiner deux conteneurs complementaires : un ConstraintLayout pour structurer l'ecran avec une Guideline, et un TableLayout pondere pour un clavier aux touches de taille egale. Il a aussi mis en pratique le View Binding, les adaptateurs de Spinner, la factorisation des ressources et la validation de la saisie. L'application respecte les quatre cas de test et toutes les regles de gestion du sujet.

Pistes d'amelioration : conversion automatique des qu'un chiffre est tape ou qu'une devise change, recuperation des taux reels via une API avec repli sur les taux fixes, et conservation du montant et des devises apres rotation grace a un ViewModel.